📘 Conditional & Loop - Flow of Control

1. Introduction to Flow of Control

Definition: Flow of control refers to the order in which statements are executed in a program.

Types of Flow

- **Sequential Flow:** Statements executed line by line in order.
- **Conditional Flow:** Statements executed based on conditions (if/else).
- **Iterative Flow:** Statements repeated multiple times (loops).

Use of Indentation

- **Purpose:** Python uses indentation to define blocks of code.
- **Rule:** All statements in a block must have the same indentation.
- **Standard:** 4 spaces or 1 tab (consistent throughout).

```
# Correct indentation
if x > 5:
    print("x is greater")  # 4 spaces
    print("This is also in if block")
print("This is outside if")  # No indentation
```

🔑 **Key Points:**

- No braces {} in Python like C/C++.
- Improper indentation causes `IndentationError`.
- Consistent indentation is crucial.

---

2. Types of Statements in Python

Python statements are classified into three main categories based on their structure and purpose.

a) Empty Statement – `pass`

Purpose: Does nothing. Used as a placeholder where syntax requires a statement but no action is needed.

```
# Example: placeholder for future code
if condition:
    pass  # TODO: implement later

# Empty class or function
class MyClass:
    pass

def my_function():
    pass
```

🔑 **Key Points:**

- No operation is performed.
- Prevents `IndentationError` when a block is required.
- Commonly used in stubs, abstract methods, or during development.

b) Simple Statement

Purpose: A single line of code that performs an action. Examples: variable assignment, print, function call, return, etc.

```
# Variable declaration
name = "Alice"
age = 25

# Print statement
print("Hello, World!")

# Function call
result = max(10, 20)

# Return statement
def add(a, b):
    return a + b
```

🔑 **Key Points:**

- Usually ends with a newline.

- Can be written on a single line.
- Does not contain indented blocks.

c) Compound Statement

Purpose: Groups multiple statements into a block. Contains a header line (ending with colon) and an indented body.

```
# Conditional statement
if score >= 90:
    grade = "A"
    print("Excellent!")

# Loop
for i in range(3):
    print(i)
    print(i * i)

# Function definition
def greet(name):
    message = "Hello, " + name
    return message
```

🔑 **Key Points:**

- Header: `if condition:` , `for x in y:` , `def func():` etc.
- Body: one or more indented statements.
- Can contain nested compound statements.

Comparison of Statement Types

Statement Type	Example	Characteristics
Empty (`pass`)	`pass`	Placeholder, does nothing
Simple	`age = 18` `print("Hello")`	Single line, performs an action
Compound	`if x > 0:` `  print("positive")`	Header + indented block

3. Conditional Statements

a) if Statement

Purpose: Executes a block of code if condition is True.

Syntax:

```
if condition:
    statement(s)
```

Example:

```
age = 18
if age >= 18:
    print("Eligible to vote")
```

🔑 **Key Points:**

- Condition must evaluate to Boolean (True/False).
- Colon (:) after condition is mandatory.

b) if-else Statement

Purpose: Executes one block if condition is True, another if False.

Syntax:

```
if condition:
    statement(s)
else:
    statement(s)
```

Example:

```
num = 7
if num % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

c) if-elif-else Statement

Purpose: Tests multiple conditions sequentially.

Syntax:

```
if condition1:
    statement(s)
elif condition2:
    statement(s)
elif condition3:
    statement(s)
else:
    statement(s)
```

Example:

```
marks = 85
if marks >= 90:
    grade = "A"
elif marks >= 80:
    grade = "B"
elif marks >= 70:
    grade = "C"
else:
    grade = "D"
print(f"Grade: {grade}")
```

d) Nested if

Purpose: if statement inside another if statement.

Syntax:

```
if condition1:
    if condition2:
        statement(s)
    else:
        statement(s)
else:
    statement(s)
```

Example:

```
age = 25
income = 50000
if age >= 18:
    if income > 30000:
        print("Eligible for loan")
    else:
        print("Low income")
else:
    print("Underage")
```

Flowchart Symbols

Start/Stop	● (Oval)	Beginning / End of program
Input/Output	▱ (Parallelogram)	Data entry / display
Process	▭ (Rectangle)	Calculations / assignments
Decision	◇ (Diamond)	Condition checking
Flow Lines	→ (Arrows)	Direction of flow

Example Programs with Conditional Statements

1. Absolute Value

```
num = -5
if num < 0:
    abs_value = -num
else:
    abs_value = num
print(f"Absolute value: {abs_value}")
```

2. Sort 3 Numbers

```
a, b, c = 15, 7, 12
if a > b:
    a, b = b, a
if a > c:
    a, c = c, a
if b > c:
    b, c = c, b
print(f"Sorted: {a}, {b}, {c}")
```

3. Divisibility Check

```
num = 15
if num % 3 == 0 and num % 5 == 0:
    print("Divisible by both 3 and 5")
elif num % 3 == 0:
    print("Divisible by 3")
elif num % 5 == 0:
    print("Divisible by 5")
else:
    print("Not divisible by 3 or 5")
```

---

4. Iterative Statements

a) for Loop

Purpose: Iterates over a sequence or range.

Method 1: Direct Iteration over Sequence Types

```
# String
for char in "Python":
    print(char)

# List
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Tuple
numbers = (1, 2, 3)
for num in numbers:
    print(num * 2)

# Dictionary
student = {"name": "Rahul", "age": 17, "class": "11"}
for key in student:      # iterates over keys
    print(f"{key}: {student[key]}")

for key, value in student.items():  # key-value pairs
    print(f"{key}: {value}")
```

Method 2: Using range() Function

📌 **range() Function Overview:**

- **Purpose:** Generates a sequence of numbers that can be used for iteration, particularly with for loops.
- **Returns:** An immutable sequence of numbers (range object).
- **Memory Efficient:** Generates numbers on-the-fly, doesn't store entire sequence in memory.

📝 **Parameters:**

- 3 Parameters taken: `range(start, stop, step)`
- **range(stop)** — *Single parameter (compulsory)*
  - `stop` : Required. Generates numbers from 0 to `stop-1` .
  - Default `start` = 0, default `step` = 1.
- **range(start, stop)** — *Two parameters*
  - `start` : Required. Starting number (included).
  - `stop` : Required. Ending number (excluded).
  - Default `step` = 1.
- **range(start, stop, step)** — *Three parameters*
  - `start` : Required. Starting number (included).
  - `stop` : Required. Ending number (excluded).
  - `step` : Required. Increment/decrement value. Can be positive or negative.

⚡ **Use of range():**

- **Primary Use:** Most commonly used with for loops for controlled iteration.
- **Conversion:** Can be converted to list using `list(range(...))` if needed.
- **Common Applications:**
  - Iterating a specific number of times
  - Counting down/reverse iteration (using negative step)

```
# Syntax: range(start, stop, step)

# Simple range (single parameter)
for i in range(5):    # 0 to 4 (stop value is exclusive)
    print(i)

# Range with start and stop (two parameters)
for i in range(2, 8):  # 2 to 7 (start inclusive, stop exclusive)
    print(i)

# Range with step (three parameters)
for i in range(1, 10, 2):  # 1,3,5,7,9 (step = 2)
    print(i)

# Reverse range (negative step)
for i in range(10, 0, -1):  # 10,9,8,...,1 (counting down)
    print(i)

# Converting range to list
numbers = list(range(5))  # [0, 1, 2, 3, 4]
print(numbers)
```

⚠️ **Important Notes:**

- `stop` value is always **exclusive** (not included in the sequence).
- `start` value is always **inclusive** (included in the sequence).
- `step` incrementing/decrementing
- If `step` = 0, Python raises a `ValueError`.
- All parameters must be integers (cannot use floats).

🔑 **Key Points for "for" Loop:**

- Works with any iterable (string, list, tuple, dict & range( ) function).
- Automatically handles iteration.
- No need for increment/decrement manually.

b) while Loop

Purpose: Repeats code while condition is True.

Syntax:

```
# initialization
while condition:
    statement(s)
    # Update condition
```

Example:

```
# Print 1 to 5
i = 1
while i <= 5:
    print(i)
    i += 1

# Sum of digits
num = 123
sum_digits = 0
while num > 0:
    sum_digits += num % 10
    num //= 10
print(f"Sum of digits: {sum_digits}")
```

🔑 **Key Points:**

- 3 parts:

  ```
  initialization, condition, updation
  ```

- Condition checked before each iteration.
- Must ensure condition becomes False eventually.
- Infinite loop if condition never becomes False.

c) break and continue Statements

break Statement

Purpose: Exits the closest enclosing loop immediately.

```
# Find first even number
for num in [3, 5, 7, 8, 9, 10]:
    if num % 2 == 0:
        print(f"First even: {num}")
        break

# Infinite loop with break
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == "quit":
        break
    print(f"You entered: {user_input}")
```

continue Statement

Purpose: Skips current iteration and continues with next.

```
# Print odd numbers only
for num in range(1, 11):
    if num % 2 == 0:
        continue
    print(num)

# Skip negative numbers
numbers = [1, -2, 3, -4, 5]
for num in numbers:
    if num < 0:
        continue
    print(f"Positive: {num}")
```

d) Nested Loops

Purpose: Loop inside another loop.

```
# Multiplication table
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=" ")
    print()  # New line

# Output:
# 1 2 3
# 2 4 6
# 3 6 9
```

5. Sample Programs

1. Generating Patterns

```
# Pattern 1: Triangle
n = 5
for i in range(1, n+1):
    print("*" * i)

# Pattern 2: Number Pyramid
n = 5
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j, end=" ")
    print()
```

2. Summation of Series

```
# Sum of series: 1 + 1/2 + 1/3 + ... + 1/n
n = 5
sum_series = 0
for i in range(1, n+1):
    sum_series += 1/i
print(f"Sum = {sum_series:.2f}")
```

3. Factorial of Positive Number

```
# Using for loop
n = 5
fact = 1
for i in range(1, n+1):
    fact *= i
print(f"Factorial of {n} = {fact}")

# Using while loop
n = 5
fact = 1
i = 1
while i <= n:
    fact *= i
    i += 1
print(f"Factorial of {n} = {fact}")
```

6. Quick Reference Card

Feature	Syntax / Example
`if`	`if condition:`
`if-else`	`if condition:` `else:`

`if-elif-else`	`if cond1:`
	`elif cond2:`
	`else:`

`for` loop	`for var in sequence:`
`for` with `range()`	`for i in range(start, stop, step):`
`while` loop	`while condition:`
`break`	Exits loop immediately
`continue`	Skips current iteration
Nested loop	Loop inside another loop
Indentation	4 spaces per block
`pass` (empty)	Placeholder, does nothing
Simple statement	`age = 18`, `print()`
Compound statement	`if ...:` block, `def ...:`

7. Important Points to Remember

- ✅ Python uses indentation, not braces
- ✅ Conditions must evaluate to boolean (True/False)
- ✅ `for` loop automatically handles iteration

- ✅ `while` loop needs manual counter update
- ✅ `break` exits, `continue` skips current iteration
- ✅ `range()` generates sequences of numbers
- ✅ Strings, lists, tuples, dicts are iterable
- ✅ Nested loops run inner loop completely for each outer loop iteration
- ✅ Avoid infinite loops
- ✅ Always plan the exit condition for `while` loops
- ✅ `pass` is a valid empty statement
- ✅ Compound statements group multiple statements in a block

— End of Notes —